

Reinforcement Learning for LLM Training

Applied Deep Learning — Chapter 15

Yin Yang

Foundation Books Project

Roadmap

Why RL for LLMs?

LLM as Policy

Reward Design

PPO for LLMs

GRPO

GSPO

LoRA / QLoRA Adaptation

Practical Recipe

Debugging & Reporting

Summary

Why RL for LLMs?

The Audit Question

Reward curves can rise while answers get worse. The question is not “did reward go up?” but **did the model improve the task, or did it learn the scorer?**

- Prompt → episode start
- Generated tokens → actions
- Completed answer → trajectory
- Verifier / rubric / preference model → reward

Goal of the chapter: make a small LLM RL fine-tune precise enough to run, audit, and report.

Supervised fine-tuning Show prompt + target completion; raise probability of target tokens.
Imitation problem.

Reinforcement learning Model produces behavior, receives a scalar evaluation, changes future behavior so high-reward outcomes become more likely.

RL matters when desired properties are easier to *judge after the fact* than to specify token by token.

Running Example: Answer Tags

Prompt: "Solve $17 + 25$ and put the final answer in answer tags."

Completion: `<reasoning>17+20=37 and 37+5=42</reasoning><answer>42</answer>`

Verifier checks: final number correct? required tags present? not too long?

Compact enough that every RL object is visible at once: a problem statement, several token actions, one sequence-level judgment after the answer tags can be checked.

LLM as Policy

Policy Over Token Sequences

Prompt x , completion $y = (y_1, \dots, y_T)$, parameters θ :

$$\pi_{\theta}(y \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}).$$

Each factor is the probability of token y_t given the prompt and the prefix already generated.

Training code uses log probabilities:

$$\log \pi_{\theta}(y \mid x) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

Products of small numbers are numerically inconvenient; sums of logs are not.

Sequence-Level Reward

A scalar $R(x, y) \in \mathbb{R}$ assigned to a full completion.

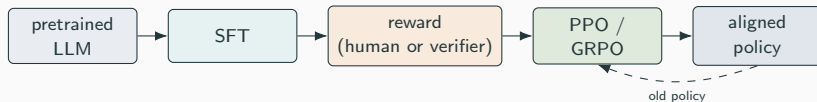
- No single token proves correctness.
- Verifier often waits until the answer tags exist.
- One scalar must be pushed back onto many token decisions.

Longer completions $\Rightarrow$ harder credit assignment, more compute, more memory, and more token decisions per scalar.

Track: max length, mean length, invalid-output rate, tokens/sec, reward components — not just the headline reward.

Reward Design

RLHF and RLVR



RLHF human judgments train a reward model.

RLVR program / verifier / parser checks the answer.

Bradley–Terry RM loss: $\mathcal{L}_{\text{RM}}(\phi) = -\log \sigma(r_{\phi}(x, y^+) - r_{\phi}(x, y^-))$.

A Component Reward

For the answer-tag example:

$$R(x, y) = 1.0 \cdot \text{exact}(x, y) + 0.2 \cdot \text{format}(y) - 0.1 \cdot \text{too_long}(y).$$

- Correct number, valid tags, not too long: $1.0 + 0.2 - 0 = 1.2$.
- Correct number, missing tags: $1.0 + 0 - 0 = 1.0$.
- Valid tags, wrong number, too long: $0 + 0.2 - 0.1 = 0.1$.

Test the scorer first. Reward bugs become training targets the moment optimization starts.

Reward Hacking

Reward gives a point whenever the output contains 42? The model learns to print 42 every answer.

- Reward checks only tags? Valid tags around nonsense.
- Reward ignores length? Correct answer + irrelevant ramble.
- Loose regex? Right number outside the tags.

Every parser edge case is part of the training signal. Save uncomfortable test cases: multiple tags, prose around the tag, empty tag, `<answer>40+2</answer>`, contradictory numbers.

PPO for LLMs

Three Policy Roles

Old policy $\pi_{\theta_{\text{old}}}$ Generated the sampled completions used in the update.

New policy π_{θ} Currently being optimized.

Reference policy π_{ref} Fixed (or slowly changing), often the SFT model.

Reference is a guardrail: KL pressure protects useful supervised behavior from an imperfect reward.

Token Probability Ratio

For one generated token,

$$r_t(\theta) = \frac{\pi_{\theta}(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}.$$

Each sampled completion should carry old-policy log probs, new-policy log probs, rewards, advantages, and KL terms so updates can be audited at token or sequence level.

Logs are evidence. Reward alone cannot explain whether clipping, KL, or probability ratios drove the update.

PPO-Style Objective

With token advantages A_t :

$$\max_{\theta} \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) - \beta \text{KL}(\pi_t^{\theta} \parallel \pi_t^{\text{ref}}) \right].$$

Three forces named:

- reward-driven advantage
- clipping (bound update size)
- reference-policy control (limit drift)

Conceptual form, completions sampled from π_θ :

$$\mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot | x)} \left[R(x, y) - \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)} \right].$$

- Strong β : reward improvement is suppressed.
- Weak β : reward-driven drift erases supervised behavior.

Report reward, KL, policy loss, value loss (if a critic is used), entropy, response length, invalid-output rate, throughput, memory.

GRPO

Group Relative Policy Optimization

Sample G completions for the *same* prompt; compare within the group. Rewards $R_1, \dots, R_G$.

Group-relative advantage:

$$A_i = \frac{R_i - \bar{R}_{\text{group}}}{s_{\text{group}} + \epsilon}.$$

Example: rewards $[0, 1, 1, 2]$, mean 1, mean-centered $[-1, 0, 0, 1]$.

No separate critic needed — the local baseline is the group mean. “Critic-free” is not “baseline-free.”

Token ratio $r_{i,t}(\theta)$ for completion i :

$$\max_{\theta} \frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \min\left(r_{i,t}(\theta)A_i, \text{clip}(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon)A_i\right) - \beta \text{KL}_{i,t}.$$

Key difference from PPO: A_i comes from the sampled group, not a critic.

What Can Go Wrong with Groups

- All four completions get reward 1? Group signal is zero.
- All four get reward 0? Same problem.
- Group size is a budget knob: more siblings $\Rightarrow$ better signal but higher generation cost.

Flat groups do not teach. GRPO needs reward variation among sibling completions for that prompt.

Record: completions per prompt, reward mean and SD, length, clipping/KL settings, invalid-output rate, tokens/sec, peak memory.

GSPO

Sequence-Level Ratio

GSPO moves the importance-ratio idea to the whole completion:

$$\rho(\theta) = \frac{\pi_{\theta}(y \mid x)}{\pi_{\theta_{\text{old}}}(y \mid x)}.$$

Two-token example: old probs 0.5, 0.1 $\Rightarrow$ 0.05. New probs 0.6, 0.1 $\Rightarrow$ 0.06. Ratio 0.06/0.05 = 1.2.

Computed in log space:

$$\log \rho_{\text{seq}}(\theta) = \sum_{t=1}^T [\log \pi_{\theta}(y_t \mid x, y_{<t}) - \log \pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})].$$

Why Sequence Level?

Imagine a long completion whose final answer is correct but whose middle reasoning drifts sharply from the old policy.

- Token-level: the noisy middle dominates the update.
- Sequence-level: one ratio summarizes the whole sampled completion — aligned with the reward unit.

Sequence-level clipped objective:

$$\max_{\theta} \frac{1}{G} \sum_{i=1}^G \min\left(\rho_i(\theta)A_i, \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon)A_i\right) - \beta \text{KL}_{\text{seq},i}.$$

GRPO vs GSPO

Question	GRPO	GSPO
Compared what?	Rewards inside a sibling group.	Probability change for a full completion.
Baseline?	Group mean (often SD-scaled).	Group-relative reward, sequence-level ratio.
Ratio unit?	Token-level ratios weighted by A_i .	One completion-level ratio from summed/averaged token log-ratios.
Failure modes	Flat groups, hacking, length bias, high invalid rate.	Unstable sequence weights, length effects, masked completions.

LoRA / QLoRA Adaptation

Adapter-Based RL Fine-Tuning

Full fine-tuning changes every weight before the reward has earned that much trust. LoRA / QLoRA: keep the base model mostly fixed, train a small adapter.

- Base supplies most of the language ability.
- Adapter carries reward-driven change.
- Rollback is mundane: discard adapter, base untouched.

Report SFT-LoRA details (rank, alpha, target modules, dropout, LR, batch, accumulation) *plus* RL knobs (completions per prompt, max length, reward weights, seed, package versions, hardware, time, tokens/s, memory).

Why an Adapter Run is Safer First

- Limits reward-driven change to the small trainable branch.
- A bad adapter is thrown away; base checkpoint preserved.
- Cheap baseline comparison: same held-out prompts, same decoding, just a different attached adapter (or none).
- Memory and storage cost much lower than full fine-tune.

Inspectable failure mode is a feature, not a bug.

Practical Recipe

Order Before Launch

1. Pick a base model that already follows the prompt format.
2. Freeze the held-out split before any reward tuning.
3. Test the reward function with no model: correct answers, wrong answers, missing tags, malformed tags, length tricks, parser bait.
4. Measure the prompt-only baseline (same template, decoding, scorer).
5. Set the training budget: max completion length, generations per prompt, LoRA rank, learning rate.

What to Record

Quantity	Why it matters	What to record
Base model	Starting behavior, memory	Checkpoint and revision
Reward function	Defines optimized behavior	Source, weights, unit tests
Prompt split	Whether evaluation is meaningful	Train/held-out, split rule
Adapter	Trainable parameters, memory	Rank, alpha, targets, quant
Generation	Reward, exploration, speed	Temp, top- p , max length, G
Measurements	Did the run help?	Held-out score, invalid rate, length, time, tokens/s, memory

A Real Tiny Run

Qwen2.5-0.5B-Instruct, 16 train prompts, 8 held-out prompts, LoRA $r = 8$.

Metric	Result
Prompt-only held-out	87.5% exact, 87.5% strict-format
RL adapter held-out	87.5% exact, 87.5% strict-format
Invalid-output rate	12.5% (GRPO and GSPO)
Avg completion length	8 tokens (max length 8)
GRPO time / throughput	59.7 s, 46.4 tok/s, 1.066 GiB
GSPO time / throughput	59.4 s, 47.1 tok/s, 1.066 GiB
Ablation effect	Token $\rightarrow$ sequence ratio: held-out unchanged

The

flat result is the result: report the negative honestly.

Debugging & Reporting

Validation Leakage

Held-out prompts influence training decisions $\Rightarrow$ final evaluation becomes too optimistic.

Common ways:

- Training accidentally on validation prompts.
- Tuning the reward after looking at validation failures.
- Choosing settings repeatedly with the same held-out set without reporting that.

Remedy: record split before training, log how often the held-out set was consulted, and say so in the report.

Invalid-Output Rate

Fraction of evaluated completions that fail a required format, parser, rubric, or safety condition *before* task correctness is considered.

12 of 100 completions miss tags? Invalid rate is 12%.

A model with high accuracy among valid outputs but high invalid rate may be unusable for the workflow.

Do not hide it inside scalar reward; report alongside it.

Three Examples in the Report

Make overclaiming hard. For the answer-tag task include:

1. One example that improved.
2. One that still failed.
3. One where the reward was misleading (high reward, wrong reasoning).

If example 3 exists: **fix the reward or parser before changing PPO, GRPO, GSPO, rank, or learning rate**. Otherwise the optimizer is just exploiting a known hole more efficiently.

Bare minimum to rerun or audit:

- model checkpoint, adapter settings
- prompt template, reward source code
- training and held-out prompt files
- decoding settings (temperature, top- p , max length)
- package versions, hardware, seed
- training time, tokens/s, peak memory
- artifact paths

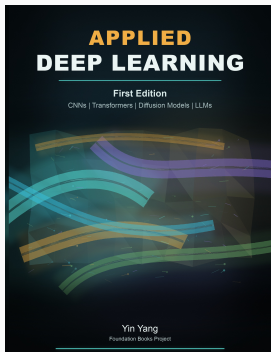
Without these, a reward curve is hard to interpret and harder to reproduce.

Summary

Summary

- LLM RL reuses the game-RL loop, but actions are tokens and reward arrives only at the completion.
- RLHF uses human feedback; RLVR uses verifiable rewards. Both fail when the reward does not measure intended behavior.
- PPO names the vocabulary: old / new / reference policy, ratio, advantage, clipping, KL. GRPO uses sibling-group baselines. GSPO uses sequence-level ratios. LoRA / QLoRA make small experiments practical.
- Reward unit tests, frozen splits, prompt-only baselines, invalid-output rates, qualitative examples, and metadata belong in the experiment plan from the start.
- Final question: **did the model improve the task, or did it learn the scorer?**

Next: Neural Network Training Revisited — distillation, compression, calibration, and the evidence package for applied training.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.